

Systems Verification

Exercise Sheet 02

Exercise 1: Checking a first model

Consider the following model:

```
byte var = 0;

active proctype A(){
  start:
  var++;
  goto start
}
```

Your goal is to verify that the variable `var` eventually reaches value 255.

Write two modified versions of the model above where:

1. The first one contains a *never claim*.
2. The second one verifies the claim using an *assertion*.

Deadline: Tuesday 8th April 2025, 8:00 a.m.

Exercise 2: Idea for an arithmetic model

In this exercise, each question is narratively expanding on the previous one. Therefore please answer each question immediately before continuing to the next one. You are encouraged to run and play around with the code in iSpin to answer the questions.

The goal is to use Spin to check a simple arithmetic property: Starting at integer value 2, multiplying continually by 2 will produce a step-wise larger integer than adding continually 2.

For this, we thought of the following code, containing the initialization of our variables a and b , our never claim and the two desired processes:

```
int a = 2;
int b = 2;

active proctype process1() {
    start:
    a = a*2;
    goto start
}
active proctype process2() {
    start:
    b = b+2;
    goto start
}
never{
    do
        :: a < b ; break ;
        :: else ;
    od
}
```

QUESTION: When initializing and running them with the following init command, will the never claim be violated or not? If so, why?

```
init { { run process1(); run process2();} }
```

QUESTION: Now, initializing with this next init command, will the never claim still be violated? If so, why did the adding of the atomic keyword not change the outcome?

```
init { atomic { run process1(); run process2();} }
```

If these experiments didn't work, surely the approach of having two separate processes that do the operations is the root of the problem, so let us just combine them into a single process, e.g. the following process3():

```
active proctype process3() {
    start:
    a = a*2;
    b = b+2;
    goto start;
}
```

Deadline: Tuesday 8th April 2025, 8:00 a.m.

QUESTION: So did it work this time (using the init call below)? If it didn't, can you explain why this model still failed?

```
init { atomic { run process3(); } }
```

Deadline: Tuesday 8th April 2025, 8:00 a.m.

Exercise 3: Some Model Coding

Write a *Promela* model containing two processes and a global **byte** variable initialised to 0 such that:

1. The first process should increment the byte variable, if it is lower than 255.
2. The second process should decrement the byte variable, if it is greater than 0.
3. The processes repeat themselves indefinitely (using the **goto** statement).

Design your model such that we can use the verifier to find out if the variable can reach value 255.

Deadline: Tuesday 8th April 2025, 8:00 a.m.